Object Oriented Programming for Data Processing
9 July 2025

Your exam material (code files, plots, datafiles, etc.) must be submitted via google classroom by 15:00 as a single `zip` file.

C++ evaluation will be based on: correct syntax, proper return types and arguments of functions, data members and class interfaces, setters/getters, correct mathematical expressions, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, comments throughout the notebook/scripts, labels, legends and plot styling and clarity in general.

Part 1 – C++

Consider the following abstract class:

```
class Function {
  public:
    Function(const std::string& name) { name_ = name; };
    // Value of the function at a point along the x-axis
    virtual double value(double x) const = 0;
    // Derivative of the function calculated at a point along the x-axis
    virtual double deriv(double x) const = 0;
    virtual std::string name() const { return name_; };
  private:
    std::string name_;
};
```

Implement a concrete class `Polynomial`, that inherits from `Function`, to represent and handle polynomials of any degree in the real variable x. The class must provide a regular:

- a constructor that takes as input a `std::vector` of `double`s that represent the coefficients of the polynomial ($a_0 + a_1 x + a_2 x^2 + ...$),
- a data member that is the degree of the polynomial,
- a setter that takes a `std::vector` as an argument, and
- a method that provides the value of the primitive at a point along the x-axis.

Implement a concrete class `Rational`, that also inherits from `Function`, to represent rational functions in the real variable x, i.e., ratios of two polynomial functions of any degree in x.

Write a short application to show how the classes works.

Part 2 – Python

Use a Python notebook or Python scripts to simulate a driven, damped, classical pendulum. The following second order ordinary differential equation governs the time-evolution of this system:

$$\frac{d^2\theta}{dt^2} = -\omega_0^2 \sin\theta - \gamma\frac{d\theta}{dt} + A\cos(\Omega t)\,,$$

where θ is the standard angular value that measures the separation from the stable equilibrium configuration, ω_0 is the pulsation of the undamped and undriven pendulum, γ regulates the viscous-type damping, and A and Ω are the amplitude and the pulsation of the drive.

The goal is to integrate this equation and simulate the pendulum evolution. To do so, the differential equation needs to be cast in a pair of first order ordinary differential equations; it is also convenient to use dimensionless variables where time is rescaled by ω_0, that is, $t' = \omega_0 t$:

$$\begin{aligned}\frac{d\theta}{dt'} &= \omega \\ \frac{d\omega}{dt'} &= -\sin\theta - \frac{\gamma}{\omega_0}\omega + \frac{A}{\omega_0^2}\cos\left(\frac{\Omega}{\omega_0}t'\right)\,.\end{aligned}$$

Provide the user with the ability to specify the system parameters and initial conditions, use the `scipy` package to integrate the equations, and provide tools to plot the results in the following. Specifically, the user should be able to display the evolution of θ and ω as functions of t' or of t, and the evolution of ω as a function of θ.

Test your code with the following setups:

- $\theta(0) = 0.1$, $\omega(0) = 0.1$, $\omega_0 = 1$, $\gamma = 0.5$, $A = 1.07$, $\Omega = 2/3$;
- $\theta(0) = 0.1$, $\omega(0) = 0$, $\omega_0 = 2$, $\gamma = 1$, $A = 4.6$, $\Omega = 4/3$.